

# Agent OS 完整架构设计研究报告

## 执行摘要

截至 **2026 年 8 月**，Agent 基础设施已经明显分层：AIOs 在研究“Agent Kernel”，Agno 在做生产 Runtime/Control Plane，Rivet 在做轻量隔离运行环境，LangGraph 在做持久化编排，SapienX 偏数字员工控制平面；与此同时，MCP 已成为 Agent↔Tool/Context 的重要协议，A2A v1.0 则面向 Agent↔Agent 互操作。<sup>1</sup>

但目前仍没有一个项目完整覆盖“**Kernel + Runtime + Security + Memory + Resource Scheduling + Protocol + Developer Platform + Multi-tenant Ops**”。这是本报告对现有项目边界的综合判断，而不是说这些项目能力不足；它们解决的是不同层。<sup>2</sup>

因此，一个真正值得长期做的 Agent OS，不应该定义成：

多 Agent 编排框架。

而应该定义为：

**异构 AI Agent 的系统软件层：统一管理 Agent 的身份、生命周期、任务、模型、工具、记忆、权限、隔离、计算资源、预算和跨 Agent 通信。**

建议最终形成五个稳定层：

```
Developer Platform
    ↓
Agent API / Control Plane
    ↓
Agent Kernel
    ↓
Runtime / Sandbox / Memory / Tool
    ↓
Models / CPU / GPU / Storage / Network / External Systems
```

最重要的战略判断是：

**真正的护城河不是“再写一个 Agent Framework”，而是建立一套 Agent 世界的系统抽象和 ABI/API。**

也就是未来不同 Agent 可以变，而：

Agent  
Task  
Permission  
ResourceQuota  
RuntimeClass  
Memory  
Tool  
IPC

这些基础对象不变。

本报告建议：**先设计 v1.0 的完整体系，再按 v0.x 逐步实现**。v0.1 不应该是“玩具 Mini Kernel”，而应是完整架构的第一条可运行纵向切片。

参考实现若采用约 **12-15 人核心团队**，合理目标是 **14-18 个月进入 v1.0**。本报告粗估研发、基础设施、安全测试和模型消耗总预算约 **430-800 万美元**；这是工程规划估算，不是市场报价，且未包括营销、销售和大型企业合规成本。目标云平台目前**未指定**，因此以下设计保持 cloud-neutral。

## 目标与总体架构

传统 OS 管的是：

Process  
Thread  
CPU  
Memory  
File  
Device  
Permission  
IPC

Agent OS 应对应为：

| 传统 OS         | Agent OS                                |
|---------------|-----------------------------------------|
| Process       | Agent / Task                            |
| Thread        | Agent Run / Subtask                     |
| CPU Scheduler | Model / Runtime Scheduler               |
| RAM           | Context / Working Memory                |
| File System   | Knowledge / Artifact / Long-term Memory |
| Device        | Tool / MCP Server                       |

| 传统 OS            | Agent OS                          |
|------------------|-----------------------------------|
| User / UID       | User + Agent Identity             |
| ACL / Capability | Permission                        |
| IPC              | Agent IPC / A2A                   |
| cgroup / quota   | ResourceQuota + Token/Cost Budget |
| syscall          | Kernel API                        |

AIOS 2024-2025 的核心贡献正是把 LLM、Memory、Storage、Tools 等资源从 Agent 应用中剥离，放进 Kernel，通过 scheduling、context management、memory management、storage management 和 access control 统一管理。其论文也说明了“Agent OS”应该首先解决**资源管理**而非 UI 或多 Agent 展示。 3

因此，我建议 Agent OS 的设计原则是：

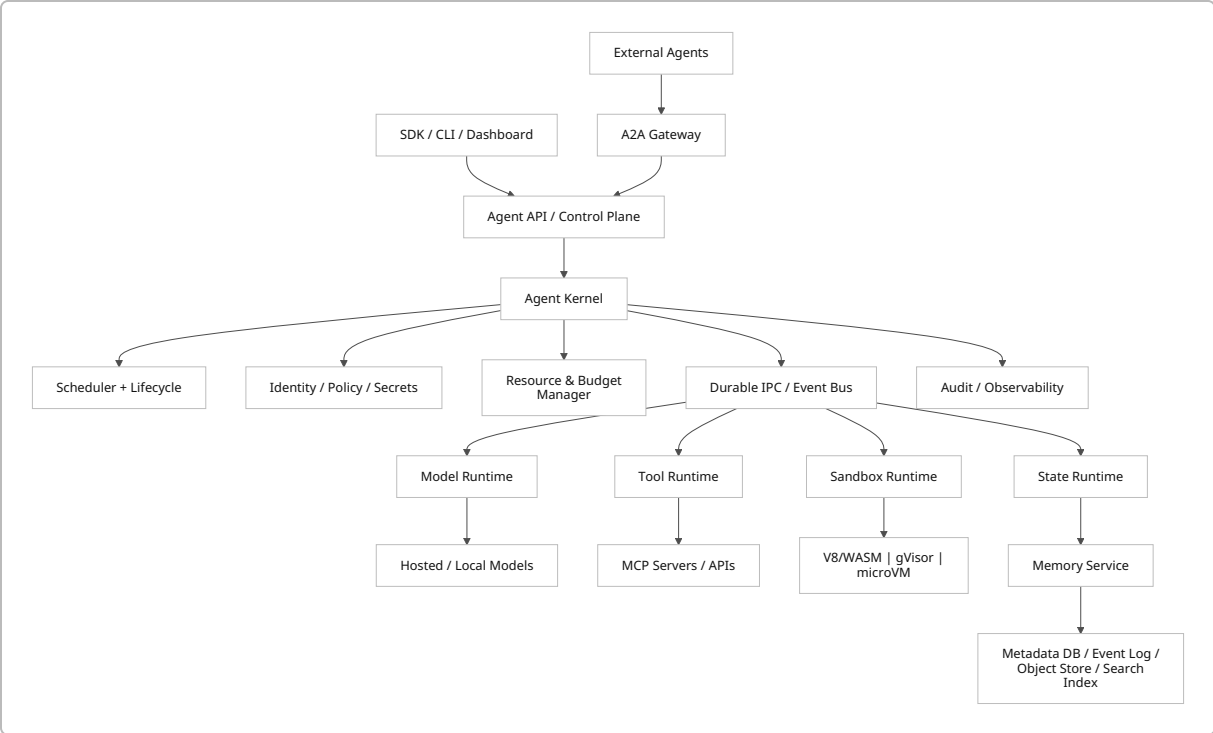
**Kernel 不理解具体业务 Agent。**

Kernel 只理解：

身份  
能力  
权限  
任务  
状态  
资源  
预算  
消息  
生命周期

Claude 类 Agent、Coding Agent、Research Agent、Browser Agent、本地 Agent、自定义企业 Agent，全部只是 Kernel 上运行的“程序”。

整体架构：



关键是明确分成两个平面：

**Control Plane**

负责 Agent 定义、Task、Scheduler、Policy、Quota、Registry、Deployment、Audit。

**Runtime/Data Plane**

真正执行模型调用、代码、浏览器、工具、Memory Retrieval 和 Agent Workload。

这和 Kubernetes 将声明式对象、调度、资源管理与实际 workload execution 分开的思想类似；Kubernetes 自身的 requests、limits 和 ResourceQuota 已证明这种资源抽象适合多租户系统。 4

**核心抽象、数据模型与 API**

我建议 Agent OS v1 最重要的工作不是先写 Scheduler，而是先冻结核心对象模型。

| 对象     | 定义           | 必须保证的系统性质         |
|--------|--------------|-------------------|
| Agent  | 可执行 AI 实体的声明 | 唯一身份、版本化          |
| Task   | 一次有目标、有预算的工作 | 持久、可恢复、幂等         |
| Kernel | 管理系统资源       | 不绑定特定模型/framework |

| 对象            | 定义                    | 必须保证的系统性质                 |
|---------------|-----------------------|---------------------------|
| Runtime       | Agent 的具体执行环境         | 可插拔                       |
| Tool          | 外部能力                  | Typed、权限控制、Side-effect 分类 |
| Memory        | Agent 可读取/写入的持久信息     | Provenance、ACL、版本         |
| Permission    | Agent 可以对什么做什么        | Default deny              |
| IPC           | Agent/Task/Runtime 消息 | Durable、traceable         |
| ResourceQuota | Agent 可消费的系统资源        | 可计量、可强制终止                 |
| Task Model    | 工作执行状态机               | crash-safe                |

## Agent 数据模型

建议从第一天就采用类似 Kubernetes 的 **desired-state manifest**，而不是把 Agent 写死成 Python class。

```

apiVersion: agentos.dev/v1alpha1
kind: Agent

metadata:
  name: research-agent
  namespace: acme
  version: "1.3.0"
  labels:
    team: research

spec:
  runtimeClass: gvisor

  capabilities:
    - research
    - filesystem.read

  model:
    policy: quality-cost-router
    allow:
      - provider-a/model-x
      - local/model-y

  tools:
    - ref: mcp://github
    - ref: mcp://browser

  memory:
    working:

```

```

    maxTokens: 32000
  longTerm:
    enabled: true
    indexes:
      - keyword
      - vector
    retentionDays: 365

  permissions:
    grants:
      - resource: "tool:github/repos/*"
        actions: [read]

  resources:
    cpu: "2"
    memory: 4Gi
    gpu: 0
    concurrency: 2
    tokenBudgetPerTask: 200000
    costBudgetUsdPerDay: 50
    networkEgress:
      - api.github.com

  lifecycle:
    restartPolicy: on-failure
    maxRetries: 3
    checkpointEvery: 30s

```

这里非常重要的一点是：

**Model 不是 Agent。**

Agent 应该可以换模型而身份不变；否则未来任何模型升级都会污染 Agent 生命周期。

## Task 数据模型

Task 应该成为 Agent OS 中与 Agent 同级的“一等公民”。

```

apiVersion: agentos.dev/v1alpha1
kind: Task

metadata:
  id: task_01J...
  namespace: acme

spec:

```

```
agentRef: research-agent@1.3.0

goal: >
  调研竞争对手并形成技术报告

input:
  artifacts:
    - artifact://brief.pdf

dependsOn: []

priority: 70
deadline: "2026-09-01T18:00:00Z"

budget:
  tokens: 200000
  costUsd: 20
  toolCalls: 500
  wallTime: 2h

approvals:
  - when: "side_effect == high"

retryPolicy:
  maxAttempts: 3
  backoff: exponential

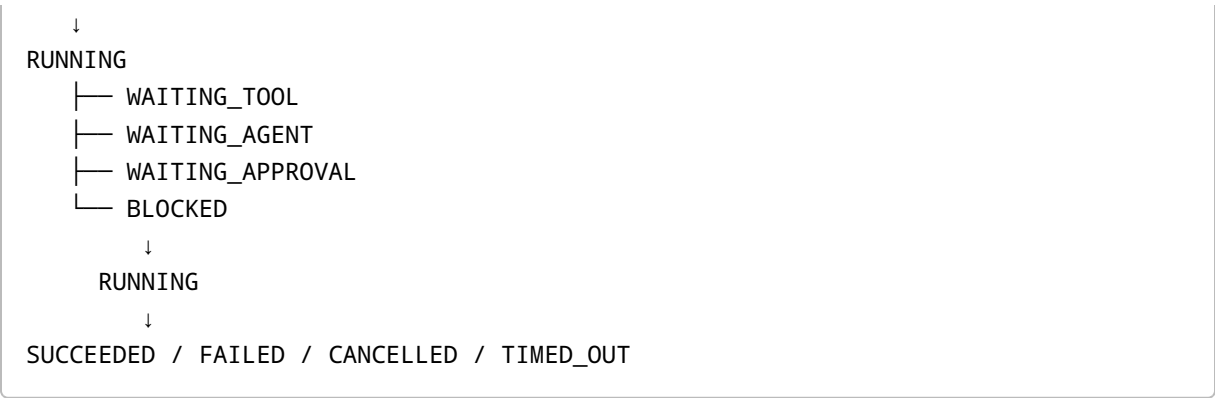
idempotencyKey: "research-acme-001"

status:
  phase: QUEUED
  attempt: 0
  assignedRuntime: null
  checkpoint: null
  usage:
    tokens: 0
    costUsd: 0
  result: null
```

其中 `status` 必须由 Kernel 管理，而不能由 Agent 自己随便修改。

建议 Task 状态机固定为：

```
QUEUED
↓
ADMITTED
```



LangGraph 已证明 checkpoint + durable execution 对长期 Agent 很重要：其 persistence 将短期状态保存为 checkpoint，长期数据存入 store，从而支持中断恢复和 human-in-the-loop。 5

IPC 数据模型

建议所有内部消息至少包含：

```
{
  "id": "msg_...",
  "type": "tool.result",
  "source": "agent://research-agent",
  "target": "task://task_01J...",
  "taskId": "task_01J...",
  "traceId": "trace_...",
  "causationId": "msg_parent",
  "timestamp": "2026-08-13T20:00:00Z",
  "ttl": 3600,
  "idempotencyKey": "...",
  "payloadRef": "artifact://..."
}
```

这里不建议承诺所谓“真正 exactly-once”。

工程上更现实的是：

**at-least-once delivery + idempotency + durable inbox/outbox。**

Agent API

外部 API 建议 REST/OpenAPI；实时事件使用 SSE/WebSocket。内部 worker plane 可以使用 gRPC/Protobuf。

| API             | 作用       |
|-----------------|----------|
| POST /v1/agents | 注册 Agent |

| API                           | 作用                   |
|-------------------------------|----------------------|
| GET /v1/agents/{id}           | 获取声明/状态              |
| PATCH /v1/agents/{id}         | 更新 desired state     |
| POST /v1/tasks                | 提交 Task              |
| GET /v1/tasks/{id}            | 查询 Task              |
| POST /v1/tasks/{id}/cancel    | Kill/Cancel          |
| POST /v1/tasks/{id}/resume    | 从 checkpoint 恢复      |
| POST /v1/tasks/{id}/approvals | 人工审批                 |
| GET /v1/tasks/{id}/events     | SSE event stream     |
| POST /v1/policy/check         | Permission decision  |
| POST /v1/memory/query         | Memory retrieval     |
| GET /v1/tools                 | Capability discovery |
| POST /v1/ipc/messages         | 内部/受控消息发送            |

内部 Kernel API 还应该抽象成：

```
spawn()
submit()
admit()
schedule()
grant()
revoke()
invokeTool()
appendMemory()
send()
checkpoint()
resume()
kill()
```

这一套东西比某个具体 REST URL 更重要，因为它实际上就是未来的 **Agent syscall model**。

## Kernel、Runtime、Memory 与安全

Kernel 是整个项目真正形成壁垒的地方。

**Scheduler** 不应该只是“哪个 Agent 先运行”。

它需要同时调度：

CPU  
RAM  
GPU  
LLM concurrency  
Token  
Money  
Tool QPS  
Storage  
Network  
Execution time  
Human approval capacity

建议调度分两步：

Admission Control  
↓  
Placement / Scheduling

Admission 先判断：

权限是否允许？  
预算是否够？  
租户 quota 是否够？  
指定 RuntimeClass 是否可用？  
GPU/模型是否可用？  
数据驻留要求是否满足？

然后 Scheduler 再基于：

Priority  
Fair-share  
Deadline  
Latency  
Cost  
Locality

决定在哪里运行。

这比单纯 Round Robin 更接近真正 OS。AIOS 的原始论文已经把 Agent scheduling 和 resource management 明确列为 Kernel 责任。 6

**Lifecycle Manager** 管：

```
create
start
pause
checkpoint
resume
migrate
restart
terminate
garbage-collect
```

Agent crash 后应该恢复 Task，而不是重新从 prompt 开始。

**Resource Manager** 需要同时记录物理资源和 AI 资源：

```
CPU / RAM / GPU
token/sec
tokens/task
$/task
$/day
tool calls
network bytes
storage
concurrency
```

Kubernetes 的 resource request/limit 机制可以直接借鉴其“Scheduler 根据 request 放置、runtime 根据 limit 执行限制”的思想。<sup>7</sup>

**Runtime**

不要只设计一个 Sandbox。

建议设计：

```
RuntimeClass
```

例如：

| RuntimeClass    | 场景         | 隔离 |
|-----------------|------------|----|
| trusted-process | 内部可信 Agent | 低  |

| RuntimeClass     | 场景                  | 隔离   |
|------------------|---------------------|------|
| v8-wasm          | Tool / JS Agent     | 中    |
| container-gvisor | 普通不可信代码             | 高    |
| microvm          | Coding/Browser/未知代码 | 很高   |
| gpu-worker       | 本地模型                | 专用资源 |

Rivet 已经展示了 V8 isolate + WebAssembly + brokered permissions 对 Agent Runtime 很有吸引力：当前 agentOS 将网络、文件系统、进程和环境变量能力做成细粒度权限，默认拒绝外向网络，并让凭证留在 host 侧。项目当前仍明确标记为 preview。<sup>8</sup>

对于风险更高的 workload，可以使用 gVisor 类用户态 application kernel，或者 Firecracker microVM。gVisor 提供 Linux-like userspace application kernel 作为额外隔离层；Firecracker 则专为安全、多租户的 microVM workload 设计。<sup>9</sup>

因此 Sandbox 不应该成为 Kernel 写死的技术，而应是：

Task → Security Risk → RuntimeClass → Sandbox Provider

Memory

Memory 不应该等于：

一个 vector database。

2025–2026 的研究正在逐渐把 memory 从简单 retrieval 提升到独立系统资源。A-MEM 研究动态链接、组织和演化 memory；MemOS 明确提出把 memory 当作可调度资源；AgeMem 则把 store/retrieve/update/summarize/discard 变成 Agent 可操作的 memory actions。<sup>10</sup>

建议分四层：

| Memory   | 内容              |
|----------|-----------------|
| Working  | 当前 Task/context |
| Episodic | Agent 发生过什么     |
| Semantic | 事实、知识、用户/组织信息   |
| Artifact | 文件、代码、报告、截图等    |

底层不要把 Vector DB 当 Source of Truth。

建议：

```
Canonical Store
  ↓
Event / Version
  ↓
Derived Indexes
├─ Vector
├─ Keyword
├─ Metadata
└─ Graph
```

关键 Task 状态、permission、quota、memory metadata 应强一致；vector/keyword/graph 索引可以最终一致。

Retrieval 流程建议：

```
Permission Filter
  ↓
Metadata Filter
  ↓
Keyword + Vector + Graph
  ↓
Rerank
  ↓
Context Budget
  ↓
Prompt
```

**ACL 必须在 Retrieval 之前，而不是检索之后。**

每条长期 Memory 应包含：

```
source
owner
tenant
createdAt
validFrom
validUntil
confidence
sensitivity
ACL
lineage
```

```
version
retention
```

这样才能做：

```
Forget
Rollback
Correction
Revocation
Legal deletion
```

## Security

Agent OS 的 Security 必须假设：

**LLM 本身是不可信控制器。**

Agent Security Bench 已系统化研究 Agent 中的攻击与防御问题；这类系统的攻击面已经不只是传统代码，还包括 prompt injection、tool misuse、memory poisoning 等 Agent 特有问题。 <sup>11</sup>

因此 Permission 应采用：

```
Principal
+
Action
+
Resource
+
Context
→
ALLOW / DENY
```

最重要的规则：

**默认拒绝。**

```
Agent 没有权限
≠
Prompt 告诉 Agent 不要做
```

真正权限必须由 Kernel/Runtime 执行。

Credential 也绝不能进入：

```
Prompt
Memory
Agent filesystem
Transcript
```

建议：

```
Agent
  ↓ asks
Secret Broker
  ↓
short-lived scoped credential
  ↓
Tool Runtime
```

Agent 最好只看到调用结果。

Rivet 当前把 credentials 保留在 host，Agent 只看到 binding 输入输出，这种思路值得吸收。 <sup>8</sup>

分布式 workload identity 可以借鉴 SPIFFE/SPIRE：SPIFFE ID 给 workload 唯一身份，SVID 提供可验证、短期的身份凭证，可用于 workload-to-workload authentication。 <sup>12</sup>

最终安全链应当是：

```
Human Identity
  ↓ delegation
Agent Identity
  ↓
Task Identity
  ↓
Permission
  ↓
Runtime
  ↓
Tool
```

而不是：

```
API_KEY = "..."  
Agent 随便使用
```

审计日志则应该 append-only，记录：

```
谁  
何时  
哪个 Agent  
哪个 Task  
使用了什么模型  
访问了什么 memory  
调用了什么 tool  
为什么被允许  
用了多少资源  
产生了什么 side effect
```

Observability 可以采用 OpenTelemetry。OTel 已有 GenAI semantic conventions，用于统一记录模型、token usage、tool calls 等信息，同时完整 prompt/completion 内容应当作为敏感数据选择性采集，而不是默认全量记录。 <sup>13</sup>

## 协议、开发平台与部署运维

到 2026 年，协议边界已经比 2024 年清楚很多。

建议明确：

```
Agent ↔ Tool      = MCP  
Agent ↔ Agent     = A2A  
Agent ↔ Kernel    = Agent OS API / ABI  
Kernel ↔ Runtime  = Internal Runtime Protocol
```

不要试图让一个协议承担全部职责。

MCP 当前 2026-07-28 规范定义的是 LLM application 与 external data/tools 的标准集成协议，基于 JSON-RPC，并进一步加入 stateless core、Tasks、extension framework 等能力。 <sup>14</sup>

MCP 本身也明确强调，协议不能替实现者自动解决用户授权、隐私和工具安全，因此 Agent OS 仍必须拥有自己的 Kernel-level Permission System。 <sup>15</sup>

A2A 则面向独立、甚至内部实现完全 opaque 的 Agent 系统之间的 discovery、communication 和 collaboration；A2A v1.0 已进入稳定规范阶段，并支持标准化跨 Agent 互操作。 <sup>16</sup>

因此：

**不要把内部 IPC 直接等同于 MCP 或 A2A。**

内部 IPC 还要承担：

```
task lease
heartbeat
checkpoint
quota
cancellation
backpressure
retry
resource accounting
kernel events
```

这些不是 MCP/A2A 的主要设计目标。

外部则通过 Adapter 暴露：

```
MCP Adapter
A2A Adapter
HTTP Adapter
Event/Webhook Adapter
```

## Developer Platform

一个真正 OS 不能只提供 Kernel。

至少需要：

```
Python SDK
TypeScript SDK
CLI
Dashboard
Agent Registry
Package Manager
Debugger
Trace Viewer
Local Emulator
Conformance Test Kit
```

Package 建议成为签名 OCI-like artifact：

```
agent-package/  
├─ agent.yaml  
├─ runtime.lock  
├─ permissions.yaml  
├─ tools.lock  
├─ memory.yaml  
├─ SBOM  
└─ signature
```

最重要的是**可重复部署**：

Agent A@1.4

应该明确对应：

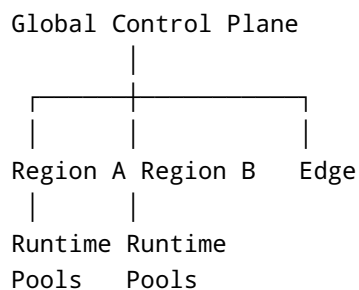
```
Agent spec version  
Runtime version  
Tool dependencies  
Model policy  
Permission policy  
Memory schema
```

否则 Agent 永远无法像正常软件一样 rollback。

## Deployment

因为目标 Cloud **未指定**，建议 v1 采用：

**Cloud-neutral Control Plane + Kubernetes reference deployment。**



Control Plane 尽量 stateless：

API Gateway  
Scheduler  
Policy  
Registry  
Controllers

状态放在：

PostgreSQL  
Event Log  
Object Store  
Memory Index  
Secret Store

worker 按 RuntimeClass 自动扩缩。

多租户模型：

Organization  
↓  
Tenant  
↓  
Namespace  
↓  
Agent  
↓  
Task

每级都应具备：

Identity  
Quota  
Policy  
Cost attribution  
Audit boundary

Kubernetes 的 namespace + RBAC + ResourceQuota 可以承担基础 infrastructure isolation，但 Agent OS 仍需额外加入 token、LLM cost、tool calls、memory 和 egress 等 AI-specific quota。 <sup>17</sup>

升级体系必须从 v0.1 开始设计：

- API versioning

Schema migration

N-1 compatibility

Runtime rolling upgrade

Canary

Agent rollback

Memory migration

Package rollback

OS 项目最危险的不是功能没写，而是早期 API 锁死以后无法升级。

现有项目对比与定位

截至 2026-08-13：

| 项目                   | 主要层                                 | 成熟度                        | License                        | 主语言                      | 隔离技术                                    | 强项                                                      | 相对完整 Agent OS 的缺口                                              |
|----------------------|-------------------------------------|----------------------------|--------------------------------|--------------------------|-----------------------------------------|---------------------------------------------------------|----------------------------------------------------------------|
| <b>AIOS</b>          | Kernel / SDK                        | 研究型 Kernel，论文已进入 COLM 2025 | 未明确；主仓库 LICENSE 当前为 0 行、1 Byte | Python 为主                | Computer-use 架构可接 VM/ MCP               | Scheduling、Memory、Storage、Tool、Access Control           | Production control plane、强隔离、多租户；维仍弱 <sup>18</sup>             |
| <b>Agno</b>          | Framework / Runtime / Control Plane | Production-oriented        | Apache-2.0                     | Python                   | 主要依赖容器/部署环境                             | REST/SSE、Postgres、Memory、RBAC、multi-tenant、A2A、MCP、OTel | 更像 Agent Platform，是低层 Kernel/ABI <sup>19</sup>                 |
| <b>Rivet agentOS</b> | Runtime / Isolation                 | Preview, 活跃开发              | Apache-2.0                     | Rust core + TS ecosystem | <b>V8 isolate + WASM</b> , 可挂外部 sandbox | 极强轻量 Runtime、权限、host binding、持久 VM                      | 分布式全局 Scheduler、Memory OS、完整 control plane 不是当前主 <sup>20</sup> |

| 项目               | 主要层                      | 成熟度                         | License                                    | 主语言                  | 隔离技术                       | 强项                                                           | 相对完整 Agent OS 的缺口                                                               |
|------------------|--------------------------|-----------------------------|--------------------------------------------|----------------------|----------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------|
| SapientX AgentOS | Operator / Control Plane | v0.7.6, 早期产品阶段              | MIT                                        | TypeScript / Next.js | 主要依赖 OpenClaw/runtime host | Worker、Task、Schedule、Approval、Runtime visibility             | Multi-user isolation、remote multi-host、长期 memory 等仍在 roadmap <sup>21</sup>      |
| LangGraph        | Orchestration Runtime    | 较成熟；2026-08-11 发布 1.2.11    | MIT                                        | Python + JS/TS       | 无内置强 sandbox               | Durable execution、Checkpoint、HITL、Memory、graph orchestration | Resource scheduler、OS permission sandbox、kernel abstraction 非主要目标 <sup>22</sup> |
| MCP              | Protocol                 | 已形成大型开放协议生态；当前规范 2026-07-28 | 新代码/规范 Apache-2.0；历史部分仍 MIT；Docs CC-BY-4.0 | TS Schema, 多语言 SDK   | 无                          | Agent/LLM ↔ Tool/Data interoperability                       | 不是 Runtime, 不负责 Scheduler/ Isolation/ Kernel Security <sup>23</sup>             |

这里最值得注意的不是谁“最好”，而是它们正在自然形成：

完整 Agent OS

AIOS

— Kernel 思想

Rivet

— Runtime / Isolation

LangGraph

— Durable Execution

Agno

— Control Plane

SapientX

— Human Operations

MCP

— Tool Protocol

A2A

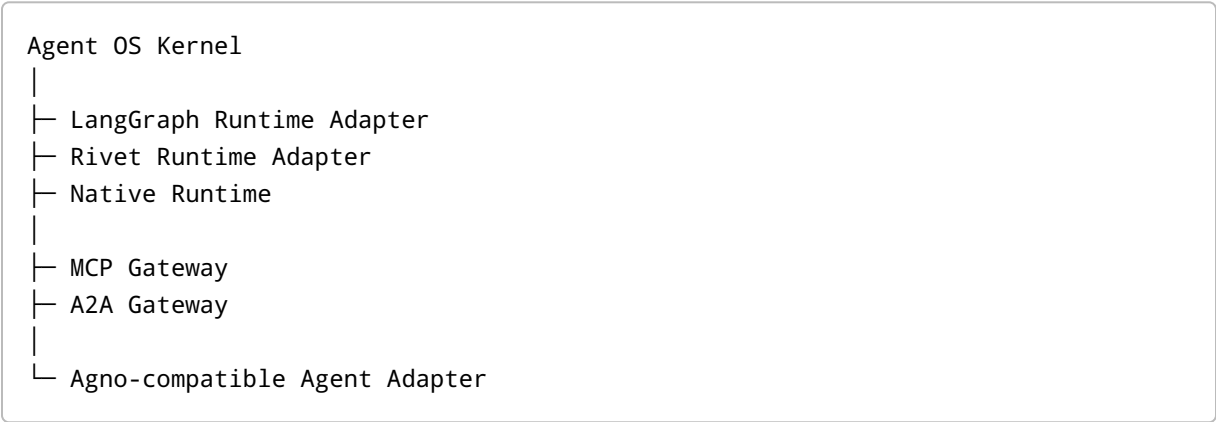
— Agent Protocol

这意味着一个新项目没有必要重写所有东西。

更聪明的做法是：

定义一个稳定 Kernel，然后把成熟项目变成 Adapter/Runtime Provider。

例如：



如此，项目竞争点就从：

“我的 Agent Framework 比别人更好。”

变成：

“不同 Agent Framework 都可以运行在我的系统之上。”

这是更有长期价值的位置。

## 路线图、团队成本与优先资料

这里所谓 **v0.x** 不是 Mini/Toy。

原则应该是：

**Architecture first, implementation incremental.**

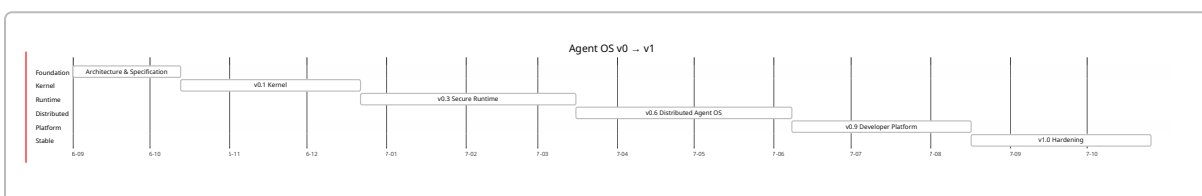
也就是说 v0.1 的实现很小，但 v1.0 的对象模型、边界和哲学从第一天就已经存在。

建议路线：

| 阶段           | 时间    | 核心结果                                                        |
|--------------|-------|-------------------------------------------------------------|
| Architecture | 0-6 周 | Agent/Task/Permission/Quota/IPC spec、Threat Model、ADR       |
| v0.1 Kernel  | 2-4 月 | Task state machine、lifecycle、scheduler、IPC、quota、checkpoint |

| 阶段                  | 时间      | 核心结果                                                            |
|---------------------|---------|-----------------------------------------------------------------|
| v0.3 Secure Runtime | 5-7 月   | V8/WASM + gVisor/microVM、Policy、Secrets、MCP/A2A                 |
| v0.6 Distributed OS | 8-10 月  | Distributed scheduler、HA、K8s、multi-tenant、Memory Service        |
| v0.9 Platform       | 11-13 月 | SDK、CLI、Dashboard、Registry、Package、upgrade                      |
| v1.0                | 14-18 月 | Stable API/ABI、SLO、security hardening、compatibility/conformance |

示意时间轴从 **2026 年 9 月** 开始：



**v0.1 的验收标准**不应该是“能跑一个 Demo”，而应该是：

两个不同 Agent Runtime  
 +  
 1000+ concurrent Tasks  
 +  
 crash/restart 不丢 Task  
 +  
 permission enforced outside LLM  
 +  
 token/cost quota 可强制中止  
 +  
 tool calls 有完整 audit  
 +  
 task 可 checkpoint/resume

到 v0.6，再追求：

multi-node  
 multi-tenant  
 failover  
 scheduler HA  
 runtime migration  
 distributed memory

建议团队

真正做完整 v1，我建议：

| Role                          | 人数  |
|-------------------------------|-----|
| Chief Architect / Kernel Lead | 1   |
| Kernel / Distributed Systems  | 3   |
| Runtime / Sandbox             | 2   |
| Security / Identity           | 1-2 |
| Memory / Data Infrastructure  | 2   |
| Control Plane / Backend       | 2   |
| Protocol / SDK / DevEx        | 1-2 |
| SRE / Infrastructure          | 1   |
| Dashboard                     | 1   |
| Security/Evals/QA             | 1   |

总计约：

12-15 名核心工程人员。

其中最不能缺的是：

Distributed Systems + Security + Runtime/Sandbox。

纯靠“很会调用 LLM API”的团队很难把 Agent OS 做深，因为最终最困难的问题会变成：

一致性

并发

调度

权限

隔离

恢复

资源计量

协议兼容

版本升级

这些本质上仍然是系统工程。

粗略成本

以较强的国际化系统工程团队估算：

| 项目                                          | 约成本             |
|---------------------------------------------|-----------------|
| 12-15 人 × 14-18 月                           | \$3.5M-\$6.0M   |
| Cloud / GPU / Models / Storage              | \$0.3M-\$0.8M   |
| Security / Red Team / Evals                 | \$0.2M-\$0.6M   |
| CI/CD / Observability / Test Infrastructure | \$0.1M-\$0.3M   |
| Compliance / 外部审计预留                         | \$0.2M-\$0.5M   |
| 总计                                          | 约 \$4.3M-\$8.0M |

这是本报告的工程估算，假定目标是一个真正可以向外部开发者开放的 v1.0 系统软件产品；若只是核心开源 Kernel，人数和资金可以明显下降。

但我不建议为了省钱删除架构。

应该减少：

实现范围

而不是减少：

长期架构完整性

最优先阅读的原始资料

| 优先级 | 资料                                               | 为什么值得读                                             |
|-----|--------------------------------------------------|----------------------------------------------------|
| P0  | <a href="#">AIOS: LLM Agent Operating System</a> | 当前最直接的 Agent Kernel 学术起点。 6                        |
| P0  | <a href="#">AIOS GitHub</a>                      | 看 scheduling、memory、storage、tool 的真实 Kernel 实现。 24 |
| P0  | <a href="#">MCP 2026-07-28 Specification</a>     | Tool/context 边界应直接兼容它，而不是重新发明协议。 25                |
| P0  | <a href="#">A2A v1.0</a>                         | 外部 Agent↔Agent 边界的重要标准。 26                         |

| 优先级 | 资料                                   | 为什么值得读                                                    |
|-----|--------------------------------------|-----------------------------------------------------------|
| P0  | <a href="#">Rivet agentOS</a>        | 当前最值得研究的 Agent lightweight runtime / isolation 思路之一。 8    |
| P0  | <a href="#">LangGraph</a>            | Durable task/state/checkpoint 设计的重要参考。 27                 |
| P1  | <a href="#">Agno</a>                 | Production Agent Control Plane、API、RBAC、deployment 参考。 19 |
| P1  | <a href="#">SapientX AgentOS</a>     | Human operator、digital-worker control plane 值得参考。 21      |
| P1  | <a href="#">A-MEM</a>                | Memory 动态组织与演化。 28                                        |
| P1  | <a href="#">MemOS</a>                | 把 Memory 提升为可管理/调度系统资源。 29                                |
| P1  | <a href="#">AgeMem</a>               | LTM/STM unified memory actions。 30                        |
| P1  | <a href="#">Agent Security Bench</a> | Agent threat model、安全 benchmark。 11                       |
| P1  | <a href="#">gVisor</a>               | 强隔离 RuntimeClass 设计参考。 31                                 |
| P1  | <a href="#">Firecracker</a>          | 高风险 Agent microVM isolation。 32                           |
| P1  | <a href="#">SPIFFE</a>               | Agent/workload identity 体系的重要基础。 33                       |
| P1  | <a href="#">OpenTelemetry</a>        | Trace/Metric/Event 与 GenAI observability 标准化。 34          |

最终建议的产品定位可以压缩成一句话：

**不要做“一个很强的 Agent”。也不要做“很多 Agent 的管理后台”。做一个让任何 Agent 都能够被安全地 spawn、schedule、grant、communicate、remember、checkpoint、migrate、audit、kill 的系统。**

如果这个抽象成立，那么它真正对应的就不是 ChatGPT、WorkBuddy 或某个 Agent Framework。

它对应的是：

**Linux / Kubernetes 在 Agent 时代所占据的那一层。**

1 3 6 18 AIOS: LLM Agent Operating System

[https://arxiv.org/abs/2403.16971?utm\\_source=chatgpt.com](https://arxiv.org/abs/2403.16971?utm_source=chatgpt.com)

2 24 agiresearch/AIOS: AIOS: AI Agent Operating System

[https://github.com/agiresearch/AIOS?utm\\_source=chatgpt.com](https://github.com/agiresearch/AIOS?utm_source=chatgpt.com)

4 17 Multi-tenancy

[https://kubernetes.io/docs/concepts/security/multi-tenancy/?utm\\_source=chatgpt.com](https://kubernetes.io/docs/concepts/security/multi-tenancy/?utm_source=chatgpt.com)

5 Persistence - Docs by LangChain

[https://docs.langchain.com/oss/python/langgraph/persistence?utm\\_source=chatgpt.com](https://docs.langchain.com/oss/python/langgraph/persistence?utm_source=chatgpt.com)

7 Resource Management for Pods and Containers

[https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/?utm\\_source=chatgpt.com](https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/?utm_source=chatgpt.com)

8 20 GitHub - rivet-dev/agentos: Give agents an operating system as a library. Runs in your existing backend – no sandboxes, VMs, or SaaS. Powered by WebAssembly & V8 isolates. · GitHub

<https://github.com/rivet-dev/agentos>

9 31 What is gVisor?

[https://gvisor.dev/docs/?utm\\_source=chatgpt.com](https://gvisor.dev/docs/?utm_source=chatgpt.com)

10 28 A-MEM: Agentic Memory for LLM Agents

[https://arxiv.org/abs/2502.12110?utm\\_source=chatgpt.com](https://arxiv.org/abs/2502.12110?utm_source=chatgpt.com)

11 Agent Security Bench (ASB): Formalizing and ...

[https://openreview.net/forum?id=V4y0CpX4hK&utm\\_source=chatgpt.com](https://openreview.net/forum?id=V4y0CpX4hK&utm_source=chatgpt.com)

12 SPIFFE Concepts

[https://spiffe.io/docs/latest/spiffe/concepts/?utm\\_source=chatgpt.com](https://spiffe.io/docs/latest/spiffe/concepts/?utm_source=chatgpt.com)

13 Inside the LLM Call: GenAI Observability with OpenTelemetry

[https://opentelemetry.io/blog/2026/genai-observability/?utm\\_source=chatgpt.com](https://opentelemetry.io/blog/2026/genai-observability/?utm_source=chatgpt.com)

14 23 25 Specification

[https://modelcontextprotocol.io/specification/2026-07-28?utm\\_source=chatgpt.com](https://modelcontextprotocol.io/specification/2026-07-28?utm_source=chatgpt.com)

15 Specification - Model Context Protocol

<https://modelcontextprotocol.io/specification/2026-07-28>

16 Agent2Agent (A2A) Protocol Specification

[https://a2a-protocol.org/latest/specification/?utm\\_source=chatgpt.com](https://a2a-protocol.org/latest/specification/?utm_source=chatgpt.com)

19 GitHub - agno-agi/agno: Build, run, and manage agent platforms. · GitHub

<https://github.com/agno-agi/agno>

21 GitHub - SapienXai/AgentOS: Run agents like a company. AgentOS is the native control plane for OpenClaw — manage agents, tasks, models, context, approvals, and runtime visibility from one place. · GitHub

<https://github.com/SapienXai/AgentOS>

22 27 langchain-ai/langgraph: Build resilient agents.

[https://github.com/langchain-ai/langgraph?utm\\_source=chatgpt.com](https://github.com/langchain-ai/langgraph?utm_source=chatgpt.com)

26 Agent2Agent (A2A) Protocol

[https://a2a-protocol.org/v1.0.0/?utm\\_source=chatgpt.com](https://a2a-protocol.org/v1.0.0/?utm_source=chatgpt.com)

29 MemOS: An Operating System for Memory-Augmented ...

[https://arxiv.org/html/2505.22101v1?utm\\_source=chatgpt.com](https://arxiv.org/html/2505.22101v1?utm_source=chatgpt.com)

30 Agentic Memory: Learning Unified Long-Term and Short ...

[https://arxiv.org/abs/2601.01885?utm\\_source=chatgpt.com](https://arxiv.org/abs/2601.01885?utm_source=chatgpt.com)

32 Firecracker

[https://firecracker-microvm.github.io/?utm\\_source=chatgpt.com](https://firecracker-microvm.github.io/?utm_source=chatgpt.com)

33 An overview of the SPIFFE specification

[https://spiffe.io/docs/latest/?utm\\_source=chatgpt.com](https://spiffe.io/docs/latest/?utm_source=chatgpt.com)

34 OpenTelemetry semantic conventions 1.44.0

[https://opentelemetry.io/docs/specs/semconv/?utm\\_source=chatgpt.com](https://opentelemetry.io/docs/specs/semconv/?utm_source=chatgpt.com)